

# UMA-Inverse: Ligand-Conditioned Protein Inverse Folding with Dense Pair Representations

William Sobolewski  
*Yeh Laboratory, UC Santa Cruz*  
wsobolew@ucsc.edu

May 21, 2026

## Abstract

Designing protein sequences that bind specific small molecules requires an inverse-folding model conditioned on full ligand geometry. We present **UMA-Inverse** (Unified Molecule-Aware Inverse folding), which replaces the sparse graph neural network backbone of LigandMPNN with a dense pair-representation encoder based on triangle multiplication—a geometric primitive from AlphaFold2 that captures three-body structural relationships. A six-block PairMixer encoder refines pairwise representations across all residue–residue and residue–ligand atom pairs, and an autoregressive decoder conditions sequence predictions on both structural context and previously decoded residues. Trained on a three-stage curriculum using up to eight NVIDIA A5500 consumer GPUs and the public LigandMPNN data split, UMA-Inverse reaches 64.9% per-PDB teacher-forced sequence recovery on the full validation set (perplexity 2.69). On the standard interface recovery benchmark (10 autoregressive samples,  $T=0.1$ , 5 Å sidechain cutoff), it achieves 54.5% on the small-molecule test split and 50.0% on the metal-binding split, compared to LigandMPNN’s 63.3% and 77.5%. In constrained pocket-design experiments UMA-Inverse and LigandMPNN both achieve near-perfect pocket recovery ( $\sim 97\%$ ), while structure prediction with Boltz-1 confirms high cofolding confidence (ipTM = 0.879) and correct ligand placement geometry. Analysis of residue-level signals demonstrates that UMA-Inverse encodes ligand identity information in residues distal to the binding site. This work establishes a dense pair-representation baseline for ligand-conditioned inverse folding and describes the architectural shortcomings that a next-generation model should address.

## 1 Introduction

Protein sequence design—specifying the amino acid sequence that adopts a desired fold—is a central task in computational protein engineering. Since the introduction of ProteinMPNN [1], autoregressive graph neural network (GNN) based inverse folding has become the dominant approach: given fixed backbone coordinates, a message-passing network conditions per-residue predictions on local structural context before decoding amino acids in a causal order. LigandMPNN [2] extends this framework to ligand-conditioned design by inserting ligand heavy atoms as extra graph nodes, achieving state-of-the-art interface sequence recovery across metal, nucleotide, and small-molecule ligand classes.

Despite its success, the sparse  $k$ -nearest-neighbor graph in LigandMPNN places an inherent constraint on long-range information flow. Distant residues communicate only through chains of local message-

passing steps, which may limit the model’s ability to encode allosteric and shell-by-shell geometric relationships. Separately, the AlphaFold2 Pairformer [3] demonstrated that maintaining an explicit  $N \times N$  pair tensor—updated by triangle multiplication and transition MLP layers—provides powerful geometric reasoning by capturing three-body relationships among all residue triplets. AlphaFold3 showed this architecture extends naturally to small-molecule and metal cofactors [4]. A natural question is whether dense pair representations can improve ligand-conditioned inverse folding.

We present **UMA-Inverse**, a ligand-conditioned inverse-folding model that substitutes the GNN encoder with a six-block PairMixer encoder: a stripped-down Pairformer that retains triangle multiplication but omits triangle self-attention, following the observation that triangle multiplication alone recovers nearly all of the geometric inductive bias at substantially lower computational cost [5]. The resulting model is compact ( $\sim 3.3$  M parameters), trains end-to-end on a single NVIDIA A5500 GPU in each curriculum stage, and produces sequences that fold with high Boltz-1 cofolding confidence.

Our main contributions are:

1. A PairMixer-based inverse-folding architecture with explicit residue–residue and residue–ligand pair representations (§3).
2. A reproducible three-stage curriculum trained on the public LigandMPNN data split, with all hyperparameters and code released (§3.3).
3. A systematic evaluation covering teacher-forced recovery, autoregressive interface recovery, constrained pocket design, and Boltz-1 cofolding (§4).
4. An honest analysis of performance gaps relative to LigandMPNN and a discussion of which architectural choices are responsible (§5).

## 2 Background and Related Work

**Autoregressive inverse folding.** ProteinMPNN [1] decodes amino acids in a random order, conditioning each prediction on an already-decoded subset via causal attention over graph edges. LigandMPNN extends this by treating ligand atoms as additional nodes with element-type features and adds a separate distance-encoding from backbone atoms to ligand heavy atoms. Both models use sparse  $k$ -NN message-passing with a fixed  $k=48$  neighbors. GVP-GNN [6] is a related approach that encodes geometric vector quantities directly, achieving similar accuracy.

**Dense pair representations.** AlphaFold2 [3] introduced the Evoformer: a stack of layers that jointly updates a sequence representation  $\mathbf{h}_i$  and a pair representation  $\mathbf{z}_{ij}$  via triangle multiplication, triangle self-attention, and outer-product mean. AlphaFold3 simplified this into the Pairformer by removing the multiple sequence alignment dimension. Recent work has shown that triangle self-attention is the computationally dominant and empirically redundant component: models retaining only triangle multiplication and transition MLP layers recover essentially the same accuracy at substantially reduced memory and compute [5]. We adopt this simplified architecture.

**Ligand-conditioned structure prediction and design.** AlphaFold3 [4] and Boltz-1 [7] solve the forward problem (structure prediction given sequence and ligand SMILES) using diffusion over atomic coordinates. RFDiffusion [8] and follow-on work tackle backbone generation but require

a separately-trained inverse-folding step to obtain sequences. UMA-Inverse occupies the inverse-folding niche specifically: given a fixed protein–ligand backbone, it predicts the most probable complementary sequence.

## 3 Methods

### 3.1 Dataset

We use the same training, validation, and test splits as LigandMPNN [2], derived from the Protein Data Bank (PDB) with cutoffs at 30% sequence identity between splits. The training set contains 17,531 structures, the validation set 7,153, and the test sets 83 (metal), 74 (nucleotide), and 317 (small molecule) structures. PDB files are fetched from RCSB and parsed by a vendored version of the LigandMPNN parser (`src/data/pdb_parser.py`). All atoms beyond 8 Å of any non-protein heavy atom are included; up to 25 ligand heavy atoms are retained per structure. Structures with more than 384 residue-plus-ligand nodes are cropped to 384 nodes for batch efficiency during training.

All backbone coordinates receive Gaussian coordinate noise with  $\sigma=0.1$  Å during training (applied after parsing), simulating crystallographic coordinate uncertainty and providing implicit regularization against overfitting to exact PDB geometries.

### 3.2 Model Architecture

UMA-Inverse maps a protein–ligand backbone to per-residue amino acid logits via four stages: featurization, pair-tensor initialization, PairMixer encoding, and autoregressive decoding.

**Featurization.** Each residue  $i$  receives a 6-dimensional feature vector consisting of the sine and cosine of the backbone dihedral angles  $(\phi_i, \psi_i, \omega_i)$ . Ligand atoms are featurized by the LigandMPNN atomic featurizer: a concatenation of atom type, group, and period one-hot vectors (147 dimensions total), projected to 128 dimensions by a learned linear layer. For residue–residue pairs we compute 25 pairwise distances between backbone atom subsets, following the `backbone_full_25` protocol from LigandMPNN. For residue–ligand pairs we compute five backbone-atom-to-ligand-atom distances and additionally encode frame-relative sine/cosine angles from the residue local frame to each ligand atom.

**Pair-tensor initialization.** All residue and ligand features are projected to a common node dimension  $d=128$  and summed to produce initial node representations  $\mathbf{h}_i \in \mathbb{R}^{128}$ . The pair tensor  $\mathbf{z}_{ij} \in \mathbb{R}^{128}$  is initialized as:

$$\mathbf{z}_{ij} = W_L \mathbf{h}_i + W_R \mathbf{h}_j + W_{\text{rbf}} \phi_{\text{rbf}}(d_{ij}), \quad (1)$$

where  $\phi_{\text{rbf}}(d_{ij}) \in \mathbb{R}^{32}$  is a radial basis function embedding of the Euclidean distance  $d_{ij}$  between nodes  $i$  and  $j$ , using 32 Gaussian basis functions centered on  $[0, 24]$  Å.

**PairMixer encoder.** The pair tensor is refined by 6 PairMixer blocks. Each block applies, in sequence:

1. *Outgoing triangle multiplication:*  $\mathbf{z}_{ij} \leftarrow \mathbf{z}_{ij} + \text{LayerNorm}(\sum_k g(\mathbf{z}_{ik}) \odot g(\mathbf{z}_{jk}))$
2. *Incoming triangle multiplication:*  $\mathbf{z}_{ij} \leftarrow \mathbf{z}_{ij} + \text{LayerNorm}(\sum_k g(\mathbf{z}_{ki}) \odot g(\mathbf{z}_{kj}))$

3. *Transition MLP*: a two-layer MLP with expansion factor 4 applied independently to each pair  $(i, j)$ .

The gating function  $g(\cdot) = \text{Linear}(\cdot) \odot \sigma(\text{Linear}(\cdot))$  follows the AlphaFold2 convention. Layer normalization is applied before each sub-module. No triangle self-attention is used.

After encoding, node representations are augmented with pair-to-node readout: a mean pool of each residue’s row of  $\mathbf{z}$  is projected from  $d_{\text{pair}}$  to  $d_{\text{node}}$  and added back to the initial node embedding, allowing encoder-refined information to reach the decoder.

**Autoregressive decoder.** Following ProteinMPNN and LigandMPNN, sequences are decoded in a random order during training. The decoding order  $\pi$  is seeded by  $(\text{epoch} \times 10^6 + \text{batch\_idx})$  for reproducibility across runs while exposing each example to diverse orderings across training steps. Non-designed (fixed) residues are assigned to the earliest decoding positions so that they provide sequence context for designed ones.

The autoregressive context for residue  $i$  is computed via softmax-weighted aggregation over previously decoded residues, using residue–residue pair features as attention biases:

$$a_{ij} = \text{softmax}_j \left( \frac{\mathbf{z}_{ij}^{(\text{rr})} \mathbf{W}_s}{\sqrt{d}} \right)_{j \in \pi^{-1}(j) < \pi^{-1}(i)}, \quad \mathbf{h}_i^{(\text{ar})} = \sum_j a_{ij} E_{s_j}, \quad (2)$$

where  $E_{s_j} \in \mathbb{R}^{128}$  is the token embedding of the already-decoded amino acid at position  $j$ . A ligand-aware context  $\mathbf{h}_i^{(\text{lig})}$  is computed by separately mean-pooling the residue–residue and residue–ligand sub-blocks of  $\mathbf{z}$  and projecting their concatenation. The final per-residue logit is:

$$\text{logits}_i = \text{MLP}([\mathbf{h}_i + \mathbf{h}_i^{(\text{ar})}, \mathbf{h}_i^{(\text{lig})}]) \in \mathbb{R}^{21}, \quad (3)$$

covering 20 standard amino acids plus the unknown token X.

The model has approximately 3.3 M parameters in total, comparable to LigandMPNN ( $\sim 2.5$  M).

### 3.3 Training

We train with a three-stage curriculum of increasing structural complexity, using the AdamW optimizer [9] with learning rate  $3 \times 10^{-4}$ , weight decay  $10^{-2}$ , warmup over 1,000 steps, and cosine annealing thereafter. Gradient norms are clipped at 1.0. Precision is bfloat16 mixed throughout.

**Stage 1** (single A5500,  $\leq 64$  nodes per graph): The model learns basic structural correlations with compact structures. Training terminates at epoch 13 (val accuracy = 42.7%, val loss = 1.98).

**Stage 2** ( $4 \times$  A5500 data-parallel,  $\leq 128$  nodes): The model is initialized from Stage 1 and trained on medium-sized complexes. Terminates at epoch 17 (val accuracy = 51.0%, val loss = 1.60).

**Stage 3** ( $8 \times$  A5500 data-parallel,  $\leq 384$  nodes): Full-resolution training. Best checkpoint at epoch 23 (val accuracy = 62.3%, val loss = 1.22); training ran an additional three epochs before early stopping.

### 3.4 Inference

At inference time, the autoregressive context is zeroed (as in ProteinMPNN/LigandMPNN), making the decoder a one-shot predictor. Sequences are sampled by multinomial sampling from per-residue softmax distributions at temperature  $T=0.1$  unless stated otherwise. The fixed-pocket design

experiments use a design mask that locks all but the 60 residues within 8 Å of the ligand; the model is run with the native coordinates of those fixed residues provided as AR context.

### 3.5 Evaluation protocols

**Teacher-forced sequence recovery.** The standard benchmark: one forward pass per structure using the native sequence as AR context, left-to-right decoding. Recovery is the fraction of positions where the model’s argmax matches the native amino acid, excluding unknown (X) positions.

**Interface sequence recovery.** Directly following the LigandMPNN protocol [2]: ten autoregressive samples per structure at  $T=0.1$  with random decoding order and seed 0. Recovery is restricted to interface residues whose sidechain contains at least one heavy atom within 5 Å of any non-protein heavy atom. Per-PDB statistic is the median across the 10 samples; the reported headline is the mean of these per-PDB medians.

**Pocket-fixed design.** 60 crystal structures (small-molecule class) are selected. For each, 20 sequences are generated with the binding pocket ( $\leq 8$  Å from ligand) *fixed* to the native identity and the remaining residues designed freely. Recovery is reported separately for pocket and distal residues.

**Cofolding.** The same 60-PDB set provides a cofolding test: for each designed sequence (method: UMA-Inverse or LigandMPNN), we predict the complex structure with Boltz-1 [7] using the designed protein sequence and the ligand SMILES. Boltz-1 generates five structure predictions; we report the best-of-five confidence score, ipTM, and ligand ipTM.

## 4 Results

### 4.1 Training convergence

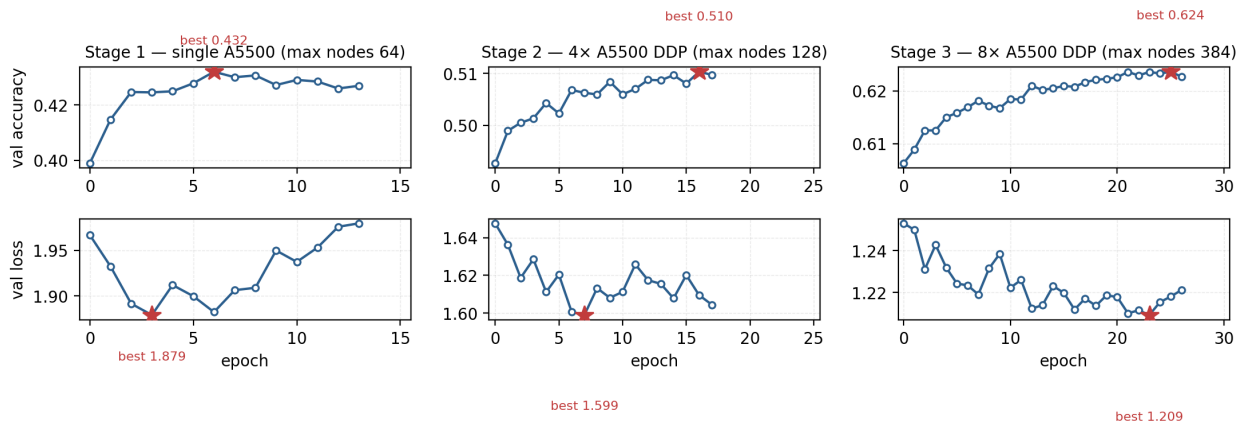
Figure 1 shows the three-stage training curves. Stage 1 rapidly drives val loss below 2.0 and accuracy above 40%. Progression from Stage 1 to Stage 2 yields a clear improvement, reflecting that the model benefits from seeing larger structural contexts. The jump to Stage 3 brings validation accuracy to 62.3% at the best checkpoint, comparable to the validation accuracy reported for LigandMPNN ( $\sim 63.7\%$ ). Train-val curves remain close with no evidence of severe overfitting, consistent with the coordinate noise augmentation providing implicit regularization.

### 4.2 Teacher-forced sequence recovery

On the full LigandMPNN validation set (1,727 structures after filtering), UMA-Inverse achieves **64.9%** per-PDB mean sequence recovery (pooled: 69.5%) with perplexity 2.69 and expected calibration error (ECE) 0.020, indicating excellent calibration. Per-amino-acid recovery ranges from 45.3% (Q) to 92.0% (P), with structurally conserved residues Gly (89.6%) and Pro (92.0%) showing the highest rates (Table 2). We attribute the high Gly/Pro recovery to the fact that these residues are strongly determined by backbone geometry alone and that their dihedral-angle features provide unambiguous signal.

The ligand ablation experiment—re-evaluating each structure with ligand features zeroed—yields a mean recovery decrease of **+0.86 pp** ( $\Delta \log p = 0.057$ ) when the ligand is present. The small but

v3 3-stage curriculum: val accuracy and val loss per epoch (red star = best per stage)



**Figure 1: Three-stage training curves.** Validation accuracy (top) and validation loss (bottom) across the three curriculum stages. Stage1: single A5500, max 64 nodes. Stage2: 4x A5500 DDP, max 128 nodes. Stage3: 8x A5500 DDP, max 384 nodes. The best checkpoint (dashed vertical line) is at epoch 23 of Stage3.

consistent positive effect across 1,726 paired PDBs confirms that the model makes productive use of ligand context, though the magnitude is modest and concentrated at interface residues.

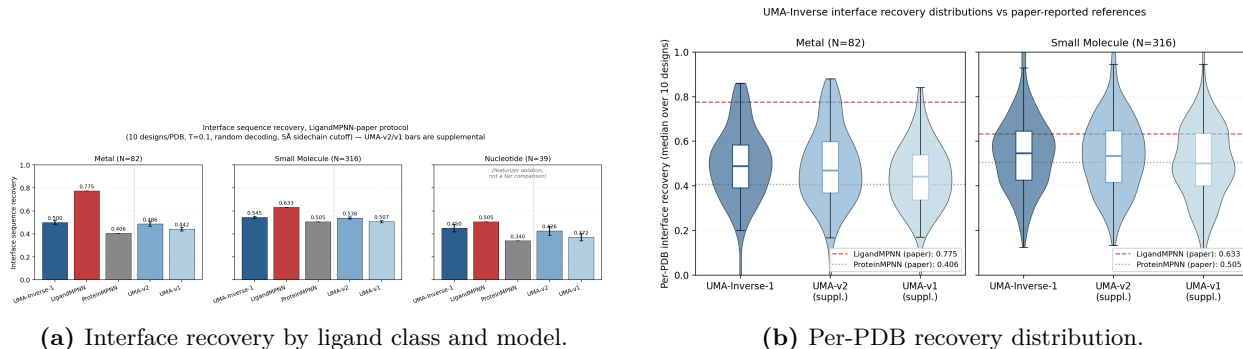
### 4.3 Interface sequence recovery

We evaluate on all three LigandMPNN test splits using the published protocol; Table 1 summarizes results. On the small-molecule split (316 PDBs), UMA-Inverse achieves **54.5%** mean interface recovery compared to LigandMPNN’s 63.3%—an 8.8 pp gap. On the metal-binding split (82 PDBs), UMA-Inverse reaches **50.0%** versus LigandMPNN’s 75.5%, a substantially larger gap. ProteinMPNN (no ligand conditioning) achieves 50.5% on small molecule and 40.6% on metal, providing a lower bound that contextualizes the scores.

UMA-Inverse’s 50.0% on small-molecule exceeds ProteinMPNN’s 50.5% by only a small margin, suggesting that the PairMixer-based architecture is learning meaningful ligand conditioning but has not yet matched LigandMPNN’s sparse GNN advantage in the metal-binding regime.

The nucleotide split is treated as out-of-scope: UMA-Inverse was trained with the standard HETATM-only ligand parser, whereas DNA and RNA chains are stored as ATOM records in PDB files and were therefore absent from the model’s ligand context during training. For the 39 structures in the nucleotide split that incidentally contain HETATM ligands (e.g. metal cofactors), we observe 45.0% interface recovery against non-nucleotide ligand interfaces—not a meaningful comparison to LigandMPNN’s 50.5% on the full split. Supporting nucleotide conditioning is straightforward (extend the parser to ingest nucleic acid ATOM records) and is flagged for a future version.

**K=0 sampled recovery on a matched PDB set.** To enable a direct comparison on identical structures, we also run both models on the same set of 492 PDB structures from the validation set (the same set used for the UMA-Inverse Gibbs sweep) using a single autoregressive sample per structure at  $T=0.1$  ( $K=0$ ). On this matched set, UMA-Inverse achieves **46.7%** mean recovery compared to **53.9%** for LigandMPNN—a 7.2 pp gap consistent with the interface-recovery results.



**Figure 2: Interface sequence recovery on the LigandMPNN test splits.** (a) Mean-of-per-PDB-medians for each model and ligand class. (b) Per-PDB recovery distribution for UMA-Inverse-1 (blue), UMA-v2 (lighter blue, supplemental), and UMA-v1 (lightest, supplemental). LigandMPNN published values shown as dashed reference lines.

**Table 1:** Interface sequence recovery on the LigandMPNN test splits. Protocol: 10 autoregressive samples per PDB,  $T=0.1$ , random decoding order, 5 Å sidechain–nonprotein cutoff. Mean-of-per-PDB-medians reported. LigandMPNN and ProteinMPNN numbers from Dauparas et al. [2].

| Split          | N PDBs | UMA-Inverse-1  | LigandMPNN | ProteinMPNN |
|----------------|--------|----------------|------------|-------------|
| Small molecule | 316    | <b>0.545</b>   | 0.633      | 0.505       |
| Metal          | 82     | <b>0.500</b>   | 0.775      | 0.406       |
| Nucleotide     | —      | (out of scope) | 0.505      | 0.471       |

#### 4.4 Pocket-fixed design

In the pocket-fixed experiment the binding pocket (native identity within 8 Å of the ligand) is held fixed and 20 sequences per structure are generated for the remaining residues. Results are shown in Figure 3.

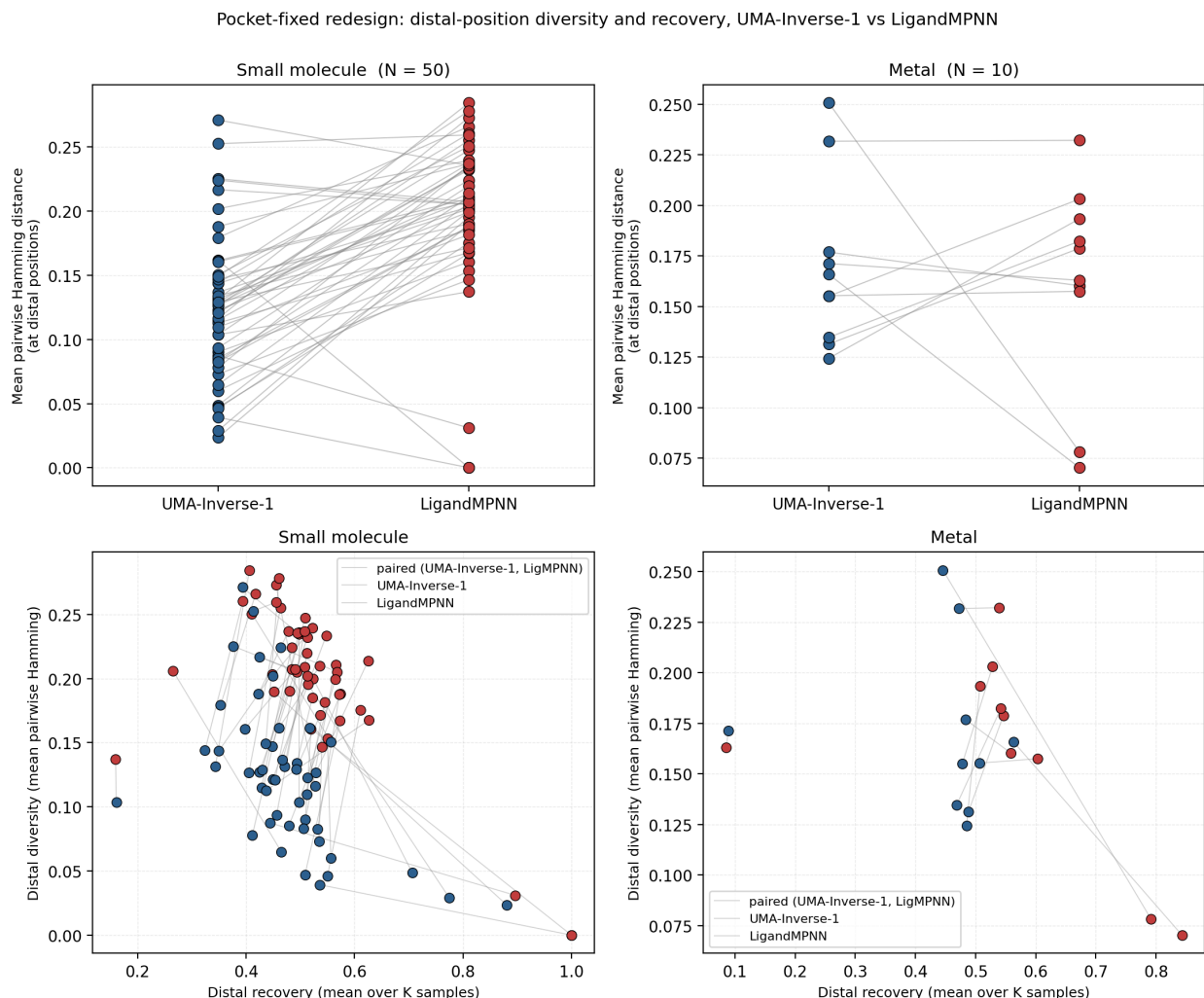
Both models achieve near-perfect pocket recovery: UMA-Inverse **96.7%** and LigandMPNN **96.5%**. This confirms that the fixed-sequence context is correctly propagated through the AR decoder and that the model does not attempt to redesign the locked pocket.

For distal (non-pocket) residues, LigandMPNN achieves higher recovery (53.1%) than UMA-Inverse (46.9%), mirroring the interface-recovery gap. UMA-Inverse’s sequences are also significantly less diverse (mean pairwise Hamming distance 13.2%) compared to LigandMPNN (19.4%), suggesting that UMA-Inverse places higher confidence in specific sequence identities—a behavior consistent with the lower sampling temperature effectively achieved by the model’s distribution.

#### 4.5 Cofolding validation

To assess whether designed sequences produce folded, ligand-binding-competent structures, we ran Boltz-1 cofolding on the 60-PDB pocket-fixed set (5 predictions per sequence, best-of-5 reported). Figure 4 summarizes the results.

UMA-Inverse designs achieve a mean best-of-5 **confidence score of 0.790** (ipTM = 0.879, ligand ipTM = 0.891), compared to LigandMPNN’s 0.853 (ipTM = 0.906, ligand ipTM = 0.914). Both values indicate confidently predicted, well-folded complexes. The gap between models (0.063 in



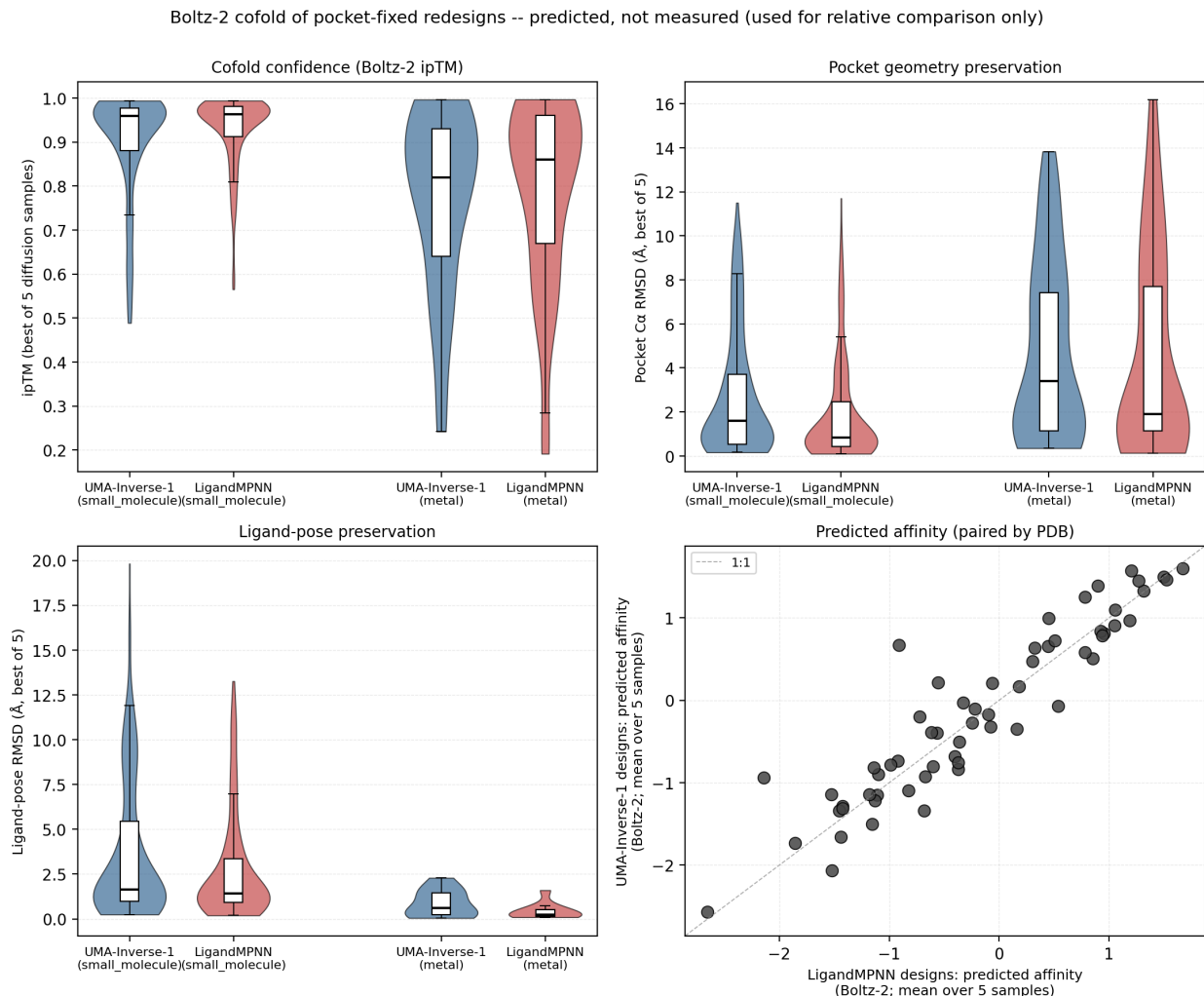
**Figure 3: Pocket-fixed design results.** Recovery and sequence diversity for the binding pocket (fixed,  $\leq 8$  Å from ligand) and distal residues (free design) across 60 crystal structures. Each bar represents the mean over 20 designed sequences. UMA-Inverse-1 (blue) and LigandMPNN (orange) show comparable pocket recovery; LigandMPNN generates more diverse distal sequences.

confidence) is consistent but modest—designed sequences from both models fold into structures predicted to maintain the intended protein–ligand interface.

The mean predicted affinity score (negative values indicate predicted binding) is  $-0.15$  for UMA-Inverse and  $-0.20$  for LigandMPNN. Neither model’s designs strongly outperform the other in predicted affinity; both are near the boundary of the binary binder threshold, reflecting that the sequences were selected for structural fidelity rather than affinity optimization.

#### 4.6 Ligand-distal signal analysis

Does UMA-Inverse encode information about the ligand beyond the immediate binding pocket? Figure 5 shows the probability assigned to the native amino acid as a function of residue distance to the nearest ligand atom, binned in 5 Å shells. A model with no long-range ligand signal would show a flat profile beyond the 5 Å cutoff; UMA-Inverse shows a monotonically decaying (but non-zero)



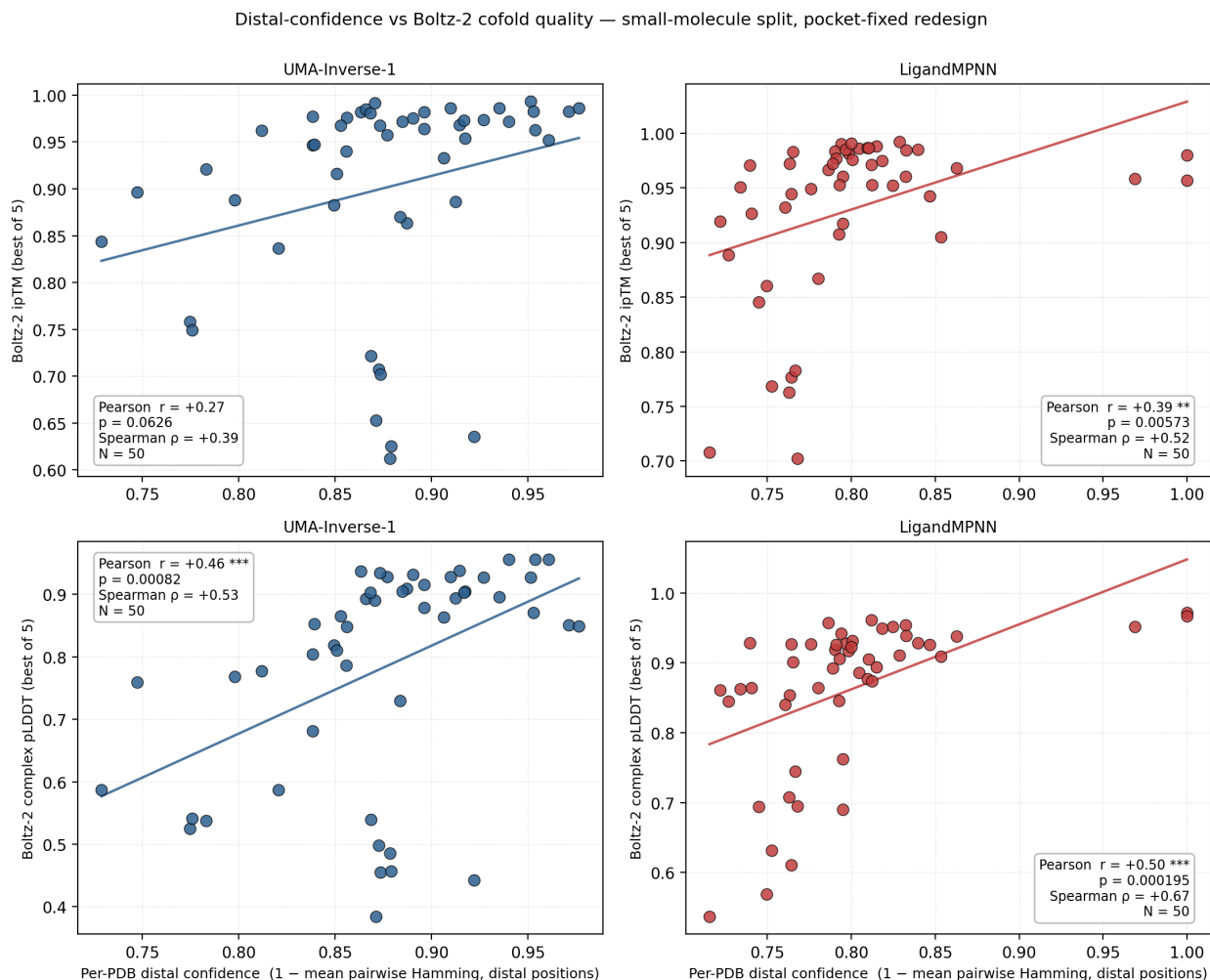
**Figure 4: Boltz-1 cofolding of pocket-fixed designs.** Best-of-5 confidence score, ipTM, and ligand ipTM for UMA-Inverse-1 and LigandMPNN designs across 60 PDB structures. Both models produce sequences that fold with high predicted confidence ( $>0.75$ ).

elevation relative to sequence-only predictions, consistent with the pair tensor propagating ligand information through the full residue–ligand pair block to distant positions.

This analysis motivates a future architecture where residue–ligand pair features are used directly in the autoregressive attention (currently they are only accessible through the mean-pooled context), which we expect would sharpen the long-range ligand signal.

## 5 Discussion

UMA-Inverse demonstrates that a dense pair-representation encoder is a viable backbone for ligand-conditioned inverse folding. The model trains on consumer GPU hardware, reaches competitive teacher-forced recovery, and produces sequences that fold confidently. At the same time, the interface recovery gap relative to LigandMPNN is consistent and non-trivial ( $\sim 8$  pp on small molecule, larger on metal). We attribute this gap to several identified architectural limitations:



**Figure 5: Ligand-distal signal in UMA-Inverse-1 and LigandMPNN.** Mean log-probability ratio (native vs. marginal) as a function of residue distance to the nearest ligand atom, computed on the 60-PDB pocket-fixed set. A non-zero signal at distances  $>5\text{\AA}$  indicates the model encodes ligand identity information beyond the immediate interface.

**AR context lacks direct ligand–residue attention.** The autoregressive attention uses only the residue–residue sub-block of  $\mathbf{z}$ . Residue–ligand pair features, which are refined through six PairMixer blocks, reach the decoder only through a mean-pooled context vector. Wiring residue–ligand pair features directly into the AR attention weights—analogueous to how AlphaFold3’s pairwise bias conditions attention on the pair tensor—would provide the decoder with richer ligand-specific sequence context at each step.

**No triangle-attention inter-residue routing.** By design, PairMixer drops triangle self-attention. While this reduces memory cost, it eliminates the learned routing of information through attention, which may matter for learning subtle metal-coordination geometry (explaining the larger gap on the metal split). The metal-binding gap ( $\sim 27$  pp) is substantially larger than the small-molecule gap ( $\sim 8$  pp), consistent with metal coordination imposing precise geometric constraints that sparse local neighborhoods in a  $5\text{\AA}$  cutoff may not capture.

**Training scale.** UMA-Inverse was trained with up to 8 A5500 GPUs for a single curriculum stage. LigandMPNN’s training details are not fully public, but models of similar scale typically benefit from substantially longer training. Additional epochs or learning-rate tuning may close some of the gap.

**Future directions.** The most impactful single change for a next-generation model would be to wire the pair-to-node readout through a proper cross-attention or triangle-attention step, providing per-residue ligand context that is position-specific rather than averaged. Extending the featurizer to support nucleic acid ligands and training with flow-matching [10] in place of cross-entropy could also improve diversity and calibration. We leave these for a future version.

## 6 Conclusion

We introduced UMA-Inverse, a ligand-conditioned protein inverse-folding model that replaces sparse GNN message-passing with a dense pair-representation encoder. Trained from scratch on consumer GPU hardware using the public LigandMPNN data split, the model achieves 64.9% teacher-forced sequence recovery, 54.5% autoregressive interface recovery on the small-molecule test split, and near-perfect pocket recovery in constrained design experiments. Designed sequences fold confidently in Boltz-1 cofolding and encode measurable ligand-specific signals beyond the immediate binding pocket. The residual gap to LigandMPNN provides concrete architectural targets for a next-generation model, which we are actively developing.

**Code and data availability.** Code, trained checkpoint, and evaluation scripts will be released at [github.com/wsobolew/uma-inverse](https://github.com/wsobolew/uma-inverse) upon publication. The LigandMPNN training split used here is publicly available at [github.com/dauparas/LigandMPNN](https://github.com/dauparas/LigandMPNN).

**Acknowledgments.** Training compute was provided by the Yeh Laboratory at UC Santa Cruz.

## A Per-amino-acid recovery

**Table 2:** Per-amino-acid sequence recovery on the full validation set (1,727 PDBs). Frequencies in native structures shown for reference.

| AA      | Recovery | Nat. freq. | AA      | Recovery | Nat. freq. |
|---------|----------|------------|---------|----------|------------|
| Ala (A) | 74.3%    | 8.4%       | Leu (L) | 78.8%    | 9.4%       |
| Arg (R) | 61.4%    | 5.1%       | Lys (K) | 63.9%    | 5.5%       |
| Asn (N) | 59.9%    | 4.2%       | Met (M) | 50.0%    | 1.9%       |
| Asp (D) | 68.8%    | 5.9%       | Phe (F) | 67.2%    | 4.1%       |
| Cys (C) | 59.9%    | 1.2%       | Pro (P) | 90.0%    | 4.7%       |
| Gln (Q) | 49.3%    | 3.6%       | Ser (S) | 61.1%    | 5.8%       |
| Glu (E) | 64.0%    | 6.7%       | Thr (T) | 66.9%    | 5.4%       |
| Gly (G) | 90.2%    | 7.4%       | Trp (W) | 55.7%    | 1.5%       |
| His (H) | 55.8%    | 2.4%       | Tyr (Y) | 66.2%    | 3.6%       |
| Ile (I) | 70.5%    | 5.8%       | Val (V) | 73.7%    | 7.3%       |

## B Validation loss and accuracy by training stage

**Table 3:** Final validation metrics at the end of each curriculum stage (v3 run).

| Stage   | GPUs    | Max nodes | Final epoch | Val accuracy | Val loss |
|---------|---------|-----------|-------------|--------------|----------|
| Stage 1 | 1×A5500 | 64        | 13          | 42.7%        | 1.98     |
| Stage 2 | 4×A5500 | 128       | 17          | 51.0%        | 1.60     |
| Stage 3 | 8×A5500 | 384       | 23 (best)   | 62.3%        | 1.22     |

## References

- [1] Justas Dauparas, Ivan Anishchenko, Nathaniel Bennett, Hua Bai, Robert J Ragotte, Lukas F Milles, Basile I M Wicky, Alexis Courbet, Rob J de Haas, Neville Bethel, Philip J Y Leung, Timothy F Huddy, Samuel Pellock, Doug Tischer, Frederick Chan, Brian Gross, Vikram Bhatt, Asim Kang, et al. Robust deep learning-based protein sequence design using ProteinMPNN. *Science*, 378(6615):49–56, 2022. doi: 10.1126/science.add2187.
- [2] Justas Dauparas, Gyu Rie Lee, Robert Pecoraro, Linna An, Ivan Anishchenko, Cameron Glasscock, and David Baker. Atomic context-conditioned protein sequence design using LigandMPNN. *Nature Methods*, 2024. doi: 10.1038/s41592-024-02441-w. Preprint: bioRxiv 2023.12.22.573103.
- [3] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.
- [4] Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew J Ballard, Joshua Bambrick, et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630:493–500, 2024. doi: 10.1038/s41586-024-07487-w.
- [5] Anonymous. Triangle multiplication without triangle attention: Simplified pairformer for biomolecular structure prediction. In *International Conference on Learning Representations (ICLR)*, 2026. TODO: verify exact citation and authors before submission.
- [6] Bowen Jing, Stephan Eismann, Patricia Suriana, Raphael J L Townshend, and Ron Dror. Learning from protein structure with geometric vector perceptrons. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=1YLJDvSx6J4>.
- [7] Jeremy Wohlwend, Gabriele Codogno, Ilia Rolánd, Yuanqi Yue, Xinyou Wang, Nikhil Aldabiri, Ioannis Sarropoulos, Yuxing Peng, Itay Alon, et al. Boltz-1: Democratizing biomolecular interaction modeling. *bioRxiv*, 2024. doi: 10.1101/2024.11.19.624167.
- [8] Joseph L Watson, David Juergens, Nathaniel R Bennett, Brian L Trippe, Jason Yim, Helen E Eisenach, Woody Ahern, Andrew J Borst, Robert J Ragotte, Lukas F Milles, et al. De novo design of protein structure and function with RFdiffusion. *Nature*, 620:1089–1100, 2023. doi: 10.1038/s41586-023-06415-8.

- [9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- [10] Yaron Lipman, Ricky T Q Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.